

Integer Registers (IA32)

				Origin (mostly obsolete)
general purpose	%eax	%ax	%ah %al	<i>accumulate</i>
	%ecx	%cx	%ch %cl	<i>counter</i>
	%edx	%dx	%dh %dl	<i>data</i>
	%ebx	%bx	%bh %bl	<i>base</i>
	%esi	%si		<i>source index</i>
	%edi	%di		<i>destination index</i>
	%esp	%sp		<i>stack pointer</i>
	%ebp	%bp		<i>base pointer</i>
16-bit virtual registers (backwards compatibility)				

x86-64 Integer Registers

%rax	%eax
%rbx	%ebx
%rcx	%ecx
%rdx	%edx
%rsi	%esi
%rdi	%edi
%rsp	%esp
%rbp	%ebp

%r8	%r8d
%r9	%r9d
%r10	%r10d
%r11	%r11d
%r12	%r12d
%r13	%r13d
%r14	%r14d
%r15	%r15d

- Extend existing registers. Add 8 new ones.
- Make **%ebp/%rbp** general purpose

Intel x86 Processors

- **Totally dominate laptop/desktop/server market**
- **Evolutionary design**
 - Backwards compatible up until 8086, introduced in 1978
 - Added more features as time goes on
- **Complex instruction set computer (CISC)**
 - Many different instructions with many different formats
 - But, only small subset encountered with Linux programs
 - Hard to match performance of Reduced Instruction Set Computers (RISC)
 - But, Intel has done just that!
 - In terms of speed. Less so for low power.

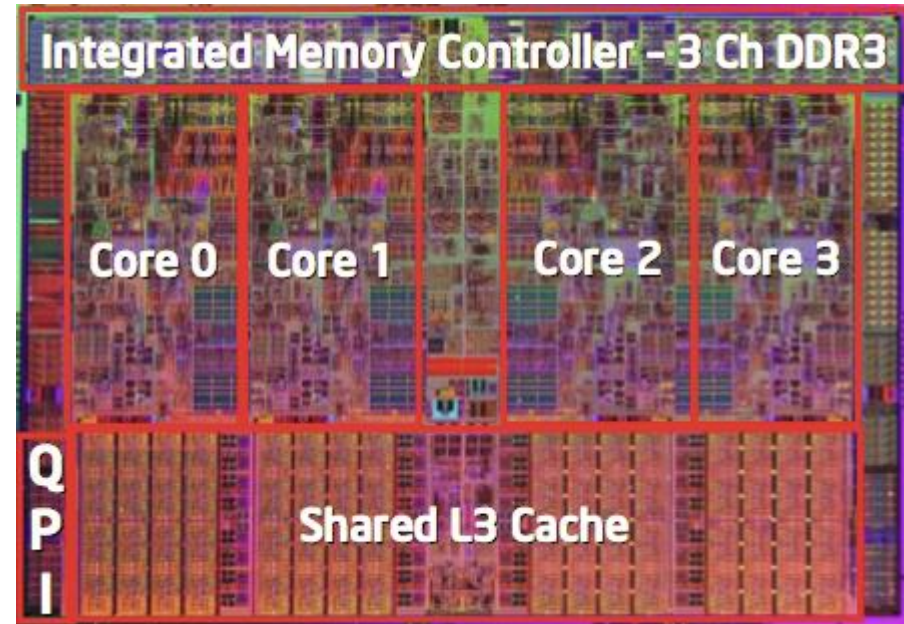
Intel x86 Evolution: Milestones

<i>Name</i>	<i>Date</i>	<i>Transistors</i>	<i>MHz</i>
■ 8086	1978	29K	5-10
▪ First 16-bit Intel processor. Basis for IBM PC & DOS ▪ 1MB address space			
■ 386	1985	275K	16-33
▪ First 32 bit Intel processor , referred to as IA32 ▪ Added “flat addressing”, capable of running Unix			
■ Pentium 4F	2004	125M	2800-3800
▪ First 64-bit Intel processor, referred to as x86-64			
■ Core 2	2006	291M	1060-3500
▪ First multi-core Intel processor			
■ Core i7	2008	731M	1700-3900
▪ Four cores (our shark machines)			

Intel x86 Processors, cont.

■ Machine Evolution

■ 386	1985	0.3M
■ Pentium	1993	3.1M
■ Pentium/MMX	1997	4.5M
■ PentiumPro	1995	6.5M
■ Pentium III	1999	8.2M
■ Pentium 4	2001	42M
■ Core 2 Duo	2006	291M
■ Core i7	2008	731M



■ Added Features

- Instructions to support multimedia operations
- Instructions to enable more efficient conditional operations
- Transition from 32 bits to 64 bits
- More cores

x86 Clones: Advanced Micro Devices (AMD)

■ Historically

- AMD has followed just behind Intel
- A little bit slower, a lot cheaper

■ Then

- Recruited top circuit designers from Digital Equipment Corp. and other downward trending companies
- Built Opteron: tough competitor to Pentium 4
- Developed x86-64, their own extension to 64 bits

Intel's 64-Bit

- **Intel Attempted Radical Shift from IA32 to IA64**
 - Totally different architecture (Itanium)
 - Executes IA32 code only as legacy
 - Performance disappointing
- **AMD Stepped in with Evolutionary Solution**
 - x86-64 (now called “AMD64”)
- **Intel Felt Obligated to Focus on IA64**
 - Hard to admit mistake or that AMD is better
- **2004: Intel Announces EM64T extension to IA32**
 - Extended Memory 64-bit Technology
 - Almost identical to x86-64!
- **All but low-end x86 processors support x86-64**
 - But, lots of code still runs in 32-bit mode

Our Coverage

■ IA32

- The traditional x86
- `shark> gcc -m32 hello.c`

■ x86-64

- The emerging standard
- `shark> gcc hello.c`
- `shark> gcc -m64 hello.c`

■ Presentation

- Book presents IA32 in Sections 3.1—3.12
- Covers x86-64 in 3.13
- We will cover both simultaneously
- Some labs will be based on x86-64, others on IA32

x86-64 Integer Registers

%rax	%eax
%rbx	%ebx
%rcx	%ecx
%rdx	%edx
%rsi	%esi
%rdi	%edi
%rsp	%esp
%rbp	%ebp

%r8	%r8d
%r9	%r9d
%r10	%r10d
%r11	%r11d
%r12	%r12d
%r13	%r13d
%r14	%r14d
%r15	%r15d

- Extend existing registers. Add 8 new ones.
- Make **%ebp/%rbp** general purpose

Instructions

- Long word l (4 Bytes) $\leftrightarrow$ Quad word q (8 Bytes)
- New instructions:
 - `movl` $\rightarrow$ `movq`
 - `addl` $\rightarrow$ `addq`
 - `sall` $\rightarrow$ `salq`
 - etc.
- 32-bit instructions that generate 32-bit results
 - Set higher order bits of destination register to 0
 - Example: `addl`

32-bit code for swap

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

```
pushl %ebp
movl  %esp,%ebp
pushl %ebx
```

} Set Up

```
movl  8(%ebp), %edx
movl  12(%ebp), %ecx
movl  (%edx), %ebx
movl  (%ecx), %eax
movl  %eax, (%edx)
movl  %ebx, (%ecx)
```

} Body

```
popl  %ebx
popl  %ebp
ret
```

} Finish

64-bit code for swap

```
void swap(int *xp, int *yp)
{
    int t0 = *xp;
    int t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

swap:

```
movl    (%rdi), %edx
movl    (%rsi), %eax
movl    %eax, (%rdi)
movl    %edx, (%rsi)
```

ret

} Set
Up

} Body

} Finish

■ Operands passed in registers (why useful?)

- First (xp) in %rdi, second (yp) in %rsi
- 64-bit pointers

■ No stack operations required

■ 32-bit data

- Data held in registers %eax and %edx
- movl operation

64-bit code for long int swap

swap_1:

```
void swap(long *xp, long *yp)
{
    long t0 = *xp;
    long t1 = *yp;
    *xp = t1;
    *yp = t0;
}
```

```
movq    (%rdi), %rdx
movq    (%rsi), %rax
movq    %rax, (%rdi)
movq    %rdx, (%rsi)
```

ret

} Set
Up

} Body

} Finish

■ 64-bit data

- Data held in registers `%rax` and `%rdx`
- `movq` operation
 - “q” stands for quad-word